

Project 11 — Reproducible Bioinformatics Pipeline

Building a reproducible workflow for biological sequence analysis

Research Question

How can a bioinformatics analysis be organized into a reproducible computational workflow rather than a collection of independent manual steps?

Project Overview

This project focuses on reproducibility and automation using a small sequence-analysis workflow. The goal is to demonstrate how data, code, results, and documentation can be connected.

Workflow

```
Input FASTA data
  ↓
Quality checking
  ↓
Sequence parsing
  ↓
Sequence statistics
  ↓
Motif / feature analysis
  ↓
Tabular results
  ↓
Summary interpretation
```

Why Reproducibility Matters

A reproducible analysis should specify input data, software or programming language, analysis steps, parameters, outputs, interpretation, and limitations. This makes it easier to repeat, inspect, debug, and extend computational work.

Python Component

```
from pathlib import Path

def read_fasta(path):
    records = []
    name, seq = None, []
    for line in Path(path).read_text().splitlines():
        line = line.strip()
        if not line:
            continue
        if line.startswith(">"):
            if name is not None:
                records.append((name, "".join(seq)))
            name = line[1:]
            seq = []
        else:
            seq.append(line.upper())
    if name is not None:
        records.append((name, "".join(seq)))
    return records
```

Project Structure

```
Project-11/
  ■■■ README.md
  ■■■ data/input.fasta
  ■■■ scripts/analysis.py
  ■■■ results/sequence_summary.csv
  ■■■ figures/workflow.png
```

Analysis Outputs

The workflow can generate sequence identifiers, sequence lengths, selected amino-acid counts, motif presence, and summary statistics in a structured table.

Limitations

This is an educational reproducibility workflow rather than a production-grade pipeline. Limitations include basic input validation, limited quality checks, no workflow manager such as Snakemake or Nextflow, and limited error handling.

What I Learned

Why reproducibility matters; how Python can automate repetitive sequence analysis; how FASTA files can be parsed programmatically; and why separating data, code, results, and documentation is useful.

Future Development

Potential extensions include multiple sequence alignment, BLAST automation, richer protein-property calculations, automated visualization, quality-control reports, and workflow management.

Conclusion

The project demonstrates a transition from manually performing individual bioinformatics tasks toward building a repeatable computational workflow.